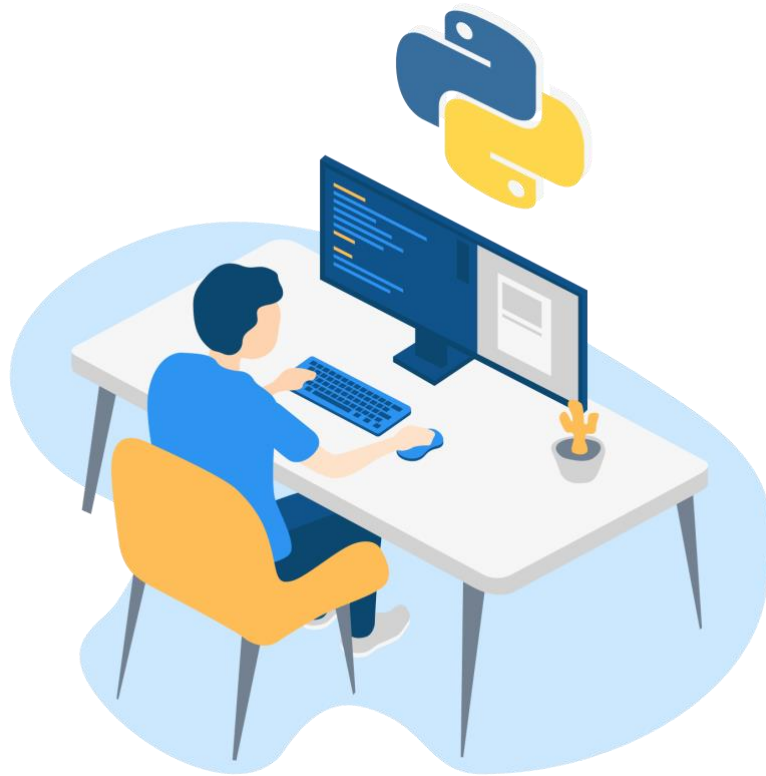


## BAB 8

### DJANGO Styling dan Form



#### 8.1 Tujuan

Setelah menyelesaikan bab ini dan mengikuti kegiatan praktikum, mahasiswa diharapkan mampu:

1. Mengelola dan menerapkan file statis (CSS) untuk menata tampilan visual aplikasi Django.
2. Mengimplementasikan konsep pewarisan template (*template inheritance*) untuk membuat layout situs yang konsisten dan efisien menggunakan `{% extends %}` dan `{% block %}`.
3. Membangun halaman detail untuk menampilkan data individual dari basis data, diakses melalui URL dinamis dengan parameter.
4. Membuat dan memproses formulir Django (`forms.ModelForm`) untuk menambah dan menyunting data melalui antarmuka web, serta menangani permintaan `GET` dan `POST`.

## 8.2 Pengantar

Pada bab sebelumnya, kita berhasil menampilkan data dari basis data ke halaman web. Namun, tampilannya masih sangat dasar dan fungsionalitasnya terbatas. Di bab ini, kita akan mempercantik aplikasi kita dengan CSS, merapikan struktur template, dan menambahkan fitur esensial yang memungkinkan pengguna untuk membuat dan mengedit konten.

### 8.2.1 Mengelola File Statis

File statis adalah aset seperti file CSS, Javascript dan gambar yang tidak berubah ketika aplikasi berjalan. Django memiliki beberapa cara untuk mengelola file tersebut :

1. Lokasi

Best practice yang sering dilakukan adalah membuat folder `static` didalam setiap direktori aplikasi. Contoh kita memiliki folder aplikasi bernama `blog`, maka kita bisa membuat folder `static` didalamnya. Misal `blog/static`.

2. Konfigurasi

Django secara otomatis akan mencari direktori `static` di dalam setiap aplikasi yang terdaftar di `INSTALLED_APPS`.

3. Menggunakan template

- Menyisipkan *template tag* `static` di bagian atas file menggunakan sintak `{% load static %}`.
- Menggunakan tag tersebut untuk menghasilkan URL ke file statis: `<link rel="stylesheet" href="{% static 'css/blog.css' %}">`. Django akan secara otomatis mencari `blog.css` di dalam direktori `static/css/` di semua aplikasi terdaftar.

### 8.2.2 Template Inheritance

Pewarisan template adalah salah satu fitur paling kuat dari sistem template Django. Dengan menggunakan template kita dapat membuat template dasar (`base.html`) yang berisi semua elemen umum dari situs kita (seperti `<html>`, `<head>`, `<body>`, `header`, `footer`) dan mendefinisikan "blok" yang dapat diisi oleh template anak.

- `{% block %}`: Di dalam template dasar, kita mendefinisikan area yang dapat diganti menggunakan tag `{% block nama_blok %}` dan `{% endblock %}`. Contoh penerapan dapat dilihat pada kode berikut.

```
1. <!-- base.html -->
2. <body>
3.     <header>...</header>
4.     <main>
5.         {% block content %}
```

```

6.         <!-- Konten default bisa diletakkan di sini -->
7.         {% endblock %}
8.     </main>
9. </body>

```

- **{% extends %}**: Di dalam template anak kita memberitahu Django untuk mewarisi dari template dasar menggunakan tag `{% extends 'blog/base.html' %}`. Tag ini harus menjadi baris pertama di template. Kemudian, kita cukup mendefinisikan ulang blok yang ingin kita isi.

```

1. <!-- post_list.html -->
2. {% extends 'blog/base.html' %}
3.
4. {% block content %}
5.     <h1>Ini adalah daftar postingan</h1>
6.     <!-- Konten spesifik untuk halaman ini -->
7. {% endblock %}

```

### 8.2.3 URL Dinamis

Seringkali kita membutuhkan halaman untuk menampilkan satu item spesifik, misalnya satu postingan blog. Ini memerlukan URL yang dinamis.

- **URL dengan Parameter**: Kita bisa menangkap bagian dari URL sebagai variabel. Contohnya, `path('post/<int:pk>/', ...)` akan menangkap sebuah integer dari URL dan menamakannya `pk` (singkatan dari *Primary Key*).
- **View dengan Argumen**: Nilai `pk` yang ditangkap dari URL akan diteruskan sebagai argumen ke fungsi `view` terkait, misalnya `def post_detail(request, pk):`.
- **get\_object\_or\_404()**: fungsi ini berguna berguna untuk menampilkan informasi 404 atau *page not found*. Fungsi ini akan mengambil sebuah objek dari basis data berdasarkan kriteria tertentu (misal: `pk`). Jika objek tidak ditemukan, ia akan secara otomatis menampilkan halaman "Page Not Found (404)".

### 8.2.4 Django Forms

Django menyediakan cara untuk pembuatan form, dari membuat form HTML, memvalidasi datanya, hingga memprosesnya.

- **ModelForm**: Cara paling efisien untuk membuat form adalah dengan `ModelForm`. `ModelForm` adalah class form yang secara otomatis membuat field-field formulir berdasarkan definisi Model Django yang kita buat.

```

1. # Di dalam file baru blog/forms.py
2. from django import forms
3. from .models import Post

```

```

4.
5. class PostForm(forms.ModelForm):
6.     class Meta:
7.         model = Post
8.         fields = ('title', 'text',)

```

- Menangani Permintaan GET dan POST:

Sebuah view yang bekerja dengan form biasanya perlu menangani dua jenis permintaan, yaitu GET dan POST:

- **GET:** Terjadi ketika pengguna baru membuka halaman. Pada tahap ini, view akan membuat form kosong (`form = PostForm()`) dan menampilkannya di halaman web.
- **POST:** Terjadi ketika pengguna sudah mengisi form dan menekan tombol submit. Data dari form tersebut dikirim ke server. View kemudian membuat form baru dengan data yang dikirim (`form = PostForm(request.POST)`), memeriksa apakah datanya valid (`form.is_valid()`), menyimpan data tersebut (`form.save()`), lalu mengarahkan pengguna ke halaman lain dengan `redirect()`.

### 8.3 Kegiatan Praktikum

#### Langkah 1: Persiapan

1. Buka kembali folder django hasil praktikum pada Bab sebelumnya
2. Aktifkan *virtual environment* pada folder tersebut tersebut.

Terminal

```
myenv\Scripts\activate.bat
```

#### Langkah 2 : Membuat URL untuk daftar Posting

1. Buat struktur direktori untuk file statis di dalam aplikasi blog: `blog/static/css/`.
2. Di dalam `blog/static/css/`, buat file baru `blog.css` dan tambahkan css berikut

`blog/static/css/blog.css`

```

1. .page-header {
2.     background-color: #C25100;
3.     margin-top: 0;
4.     margin-bottom: 40px;
5.     padding: 20px 20px 20px 40px;
6. }
7.
8. .page-header h1,

```

```

9. .page-header h1 a,
10. .page-header h1 a:visited,
11. .page-header h1 a:active {
12.     color: #ffffff;
13.     font-size: 36pt;
14.     text-decoration: none;
15. }
16.
17. h1,
18. h2,
19. h3,
20. h4 {
21.     font-family: 'Lobster', cursive;
22. }
23.
24. .date {
25.     color: #828282;
26. }
27.
28. .save {
29.     float: right;
30. }
31.
32. .post-form textarea,
33. .post-form input {
34.     width: 100%;
35. }
36.
37. .top-menu,
38. .top-menu:hover,
39. .top-menu:visited {
40.     color: #ffffff;
41.     float: right;
42.     font-size: 26pt;
43.     margin-right: 20px;
44. }
45.
46. .post {
47.     margin-bottom: 70px;
48. }
49.
50. .post h2 a,
51. .post h2 a:visited {
52.     color: #000000;
53. }
54.
55. .post > .date,
56. .post > .actions {
57.     float: right;
58. }
59.

```

```

60. .btn-secondary,
61. .btn-secondary:visited {
62.     color: #C25100;
63.     background: none;
64.     border-color: #C25100;
65.     margin-left: 15px;
66. }
67.
68. .btn-secondary:hover {
69.     color: #FFFFFF;
70.     background-color: #C25100;
71. }

```

3. Buat template dasar. Di dalam `blog/templates/blog/`, buat file baru `base.html`.

`blog/templates/blog/base.html`

```

1. {% load static %}
2. <!DOCTYPE html>
3. <html>
4.     <head>
5.         <title>Django blog</title>
6.         <link rel="stylesheet" href="
https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css">
7.         <link rel="stylesheet" href="{% static 'css/blog.css' %}">
8.         <link href="https://fonts.googleapis.com/css?family=Lobster&subset=latin,latin-ext" rel="stylesheet" type="text/css">
9.     </head>
10.    <body>
11.        <header class="page-header">
12.            <div class="container">
13.                <h1><a href="/">Django Blog</a></h1>
14.            </div>
15.        </header>
16.        <main class="container">
17.            <div class="row">
18.                <div class="col">
19.                    {% block content %}
20.                    {% endblock %}
21.                </div>
22.            </div>
23.        </main>
24.    </body>
25.</html>

```

4. Modifikasi `blog/templates/blog/post_list.html` untuk mewarisi dari `base.html`. Hapus semua kode HTML yang ada kemudian isi dengan kode berikut.

blog/templates/blog/post\_list.html

```
1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4.     {% for post in posts %}
5.         <article class="post">
6.             <h2><a href="">{{ post.title }}</a></h2>
7.             <p>Published: {{ post.published_date }}</p>
8.             <p>{{ post.text|linebreaksbr }}</p>
9.         </article>
10.    {% endfor %}
11.{% endblock %}
```

5. Jalankan server dan buka browser pada alamat <http://127.0.0.1:8000/>.

Terminal

```
|python manage.py runserver
```



### Langkah 3: Membuat Detail Halaman Postingan

1. Buka `blog/urls.py` dan tambahkan path untuk halaman detail.

blog/urls.py

```
1. from django.urls import path
2. from . import views
3.
```

```

4. urlpatterns = [
5.     path('', views.post_list, name='post_list'),
6.     path('post/<int:pk>/', views.post_detail, name='post_detail'),
7. ]

```

2. Buka `blog/views.py` dan tambahkan *view* `post_detail`.

`blog/views.py`

```

1. from django.shortcuts import render, get_object_or_404
2. from django.utils import timezone
3. from .models import Post
4.
5. # Create your views here
6. def post_list(request):
7.     posts = Post.objects.filter(published_date__lte=timezone.now())
8.     .order_by('published_date')
9.     return render(request, 'blog/post_list.html', {'posts': posts})
10.
11. def post_detail(request, pk):
12.     post = get_object_or_404(Post, pk=pk)
13.     return render(request, 'blog/post_detail.html', {'post': post})
14.

```

3. Buat file template `blog/templates/blog/post_detail.html`.

`blog/templates/blog/post_detail.html`.

```

1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4.     <article class="post">
5.         {% if post.published_date %}
6.             <time class="date">
7.                 {{ post.published_date }}
8.             </time>
9.         {% endif %}
10.        <h2>{{ post.title }}</h2>
11.        <p>{{ post.text|linebreaksbr }}</p>
12.    </article>
13. {% endblock %}

```

4. Perbarui tautan di `post_list.html` agar mengarah ke halaman detail.

`blog/templates/blog/post_list.html`.

```

1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4.     {% for post in posts %}
5.         <article class="post">
6.             <h2><a href="{% url 'post_detail' pk=post.pk %}">{{ post.title }}</a></h2>
7.             <p>Published: {{ post.published_date }}</p>
8.             <p>{{ post.text|linebreaksbr }}</p>
9.         </article>
10.     {% endfor %}
11. {% endblock %}

```

5. *Refresh* halaman utama dan klik salah satu judul postingan untuk melihat hasilnya.

#### Langkah 4: Membuat Form untuk Postingan Baru

1. Buat file baru di dalam direktori `blog` bernama `forms.py`.

`blog/forms.py`

```

1. # blog/forms.py
2. from django import forms
3. from .models import Post
4.
5. class PostForm(forms.ModelForm):
6.     class Meta:
7.         model = Post
8.         fields = ('title', 'text',)

```

2. Buka `blog/urls.py` dan tambahkan path untuk halaman postingan baru.

`blog/urls.py`

```

1. from django.urls import path
2. from . import views
3.
4. urlpatterns = [
5.     path('', views.post_list, name='post_list'),
6.     path('post/<int:pk>/', views.post_detail, name='post_detail'),
7.     path('post/new/', views.post_new, name='post_new'),
8. ]

```

3. Buka `blog/views.py` dan tambahkan *view* `post_new`.

`blog/views.py`

```

1. from django.shortcuts import render, get_object_or_404, redirect
2. from django.utils import timezone
3. from .models import Post
4. from .forms import PostForm
5.
6. # Create your views here
7. def post_list(request):
8.     posts = Post.objects.filter(published_date__lte=timezone.now())
9.     .order_by('published_date')
10.     return render(request, 'blog/post_list.html', {'posts': posts})
11.
12. def post_detail(request, pk):
13.     post = get_object_or_404(Post, pk=pk)
14.     return render(request, 'blog/post_detail.html', {'post': post})
15.
16. def post_new(request):
17.     if request.method == "POST":
18.         form = PostForm(request.POST)
19.         if form.is_valid():
20.             post = form.save(commit=False)
21.             post.author = request.user
22.             post.published_date = timezone.now()
23.             post.save()
24.             return redirect('post_detail', pk=post.pk)
25.         else:
26.             form = PostForm()
27.             return render(request, 'blog/post_edit.html', {'form': form})

```

#### 4. Buat file template `blog/templates/blog/post_edit.html`.

`blog/templates/blog/post_edit.html`.

```

1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4.     <h2>New post</h2>
5.     <form method="POST" class="post-form">{% csrf_token %}
6.         {{ form.as_p }}
7.         <button type="submit" class="save btn btn-
8.             secondary">Save</button>
9.     </form>
10. {% endblock %}

```

#### 5. Tambahkan tautan untuk membuat post baru di `base.html`.

`blog/templates/blog/base.html`.

```

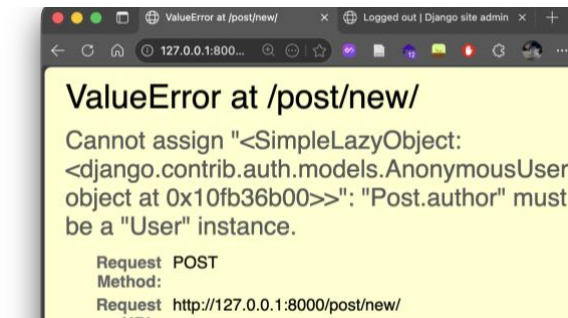
1. {% load static %}
2. <!DOCTYPE html>
3. <html>
4.     <head>
5.         <title>Django blog</title>
6.         <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css">
7.         <link rel="stylesheet" href="{% static 'css/blog.css' %}">
8.         <link href="https://fonts.googleapis.com/css?family=Lobster&subset=latin,latin-ext" rel="stylesheet" type="text/css">
9.     </head>
10.    <body>
11.        <header class="page-header">
12.            <div class="container">
13.                <a href="{% url 'post_new' %}" class="top-menu">+ New</a>
14.                <h1><a href="/">Django Blog</a></h1>
15.            </div>
16.        </header>

```

6. Refresh halaman kemudian klik +New untuk menambah post baru



7. Jika menemukan pesan error seperti berikut maka lakukan login admin terlebih dahulu. Buka halaman <http://127.0.0.1:8000/admin> kemudian login menggunakan akun admin yang sudah diatur pada Modul 6.



## Langkah 5: Membuat Form untuk Edit Postingan

1. Buka `blog/urls.py` dan tambahkan path untuk mengedit.

`blog/urls.py`

```
2. from django.urls import path
3. from . import views
4.
5. urlpatterns = [
6.     path('', views.post_list, name='post_list'),
7.     path('post/<int:pk>/', views.post_detail, name='post_detail'),
8.     path('post/new/', views.post_new, name='post_new'),
9.     path('post/<int:pk>/edit/', views.post_edit, name='post_edit')
10. ]
```

2. Buka `blog/views.py` dan tambahkan `view post_edit` pada bagian terakhir.

`blog/views.py`

```
1. ..
2. ..
3. #kode sebelumnya
4.
5. def post_edit(request, pk):
6.     post = get_object_or_404(Post, pk=pk)
7.     if request.method == "POST":
8.         form = PostForm(request.POST, instance=post)
9.         if form.is_valid():
10.            post = form.save(commit=False)
11.            post.author = request.user
12.            post.published_date = timezone.now()
13.            post.save()
14.            return redirect('post_detail', pk=post.pk)
15.     else:
16.         form = PostForm(instance=post)
17.     return render(request, 'blog/post_edit.html', {'form': form})
```

3. Tambahkan tautan untuk mengedit di `post_detail.html`.

`blog/templates/blog/post_detail.html`.

```

1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4.     <article class="post">
5.         <aside class="actions">
6.             <a class="btn btn-
secondary" href="{% url 'post_edit' pk=post.pk %}">Edit</a>
7.         </aside>
8.         {% if post.published_date %}
9.             <time class="date">
10.                 {{ post.published_date }}
11.             </time>
12.         {% endif %}
13.         <h2>{{ post.title }}</h2>
14.         <p>{{ post.text|linebreaksbr }}</p>
15.     </article>
16. {% endblock %}

```

## Langkah 5 : Menambahkan Autentikasi

1. Saat ini, tautan "New Post" dan "Edit" terlihat oleh semua orang, yang tentu saja tidak kita inginkan. Kita akan menyembunyikannya agar hanya terlihat oleh admin yang *login*. Buka file `blog/templates/blog/base.html`. Cari tautan "New Post" yang kita tambahkan pada Langkah 4. Bungkus tautan tersebut dengan tag `{% if user.is_authenticated %}`

```

1. {% if user.is_authenticated %}
2.     <a href="{% url 'post_new' %}" class="top-menu">+ New</a>
3. {% endif %}

```

2. Buka file `blog/templates/blog/post_detail.html`. Cari tautan "Edit" yang kita tambahkan pada Langkah 5. Bungkus tautan tersebut dengan tag `{% if %}` yang sama.

```

1. <aside class="actions">
2.     {% if user.is_authenticated %}
3.         <a class="btn btn-
secondary" href="{% url 'post_edit' pk=post.pk %}">Edit</a>
4.     {% endif %}
5. </aside>

```

## 8.4 Rangkuman

- File statis seperti CSS dikelola di dalam direktori `static` per aplikasi dan diakses di template menggunakan `{% load static %}` dan `{% static 'path/to/file' %}`.
- Pewarisan template dengan `{% extends %}` dan `{% block %}` adalah teknik fundamental untuk menjaga kode mudah dikelola.
- URL dinamis memungkinkan kita membuat halaman untuk item data individual (misalnya, `post/<int:pk>/`) dengan meneruskan parameter dari URL ke *view*.
- Django Forms, khususnya `ModelForm`, berfungsi menyederhanakan proses pembuatan, validasi, dan penyimpanan data dari formulir web.
- *Views* yang menangani form harus bisa membedakan antara permintaan `GET` (menampilkan form) dan `POST` (memproses data yang dikirim).
- Kita dapat menggunakan variabel `{{ user.is_authenticated }}` di dalam *template* untuk menampilkan konten secara kondisional hanya kepada pengguna yang telah *login*.

## 8.5 Tugas

Kerjakan tugas-tugas di bawah ini untuk memperkuat pemahaman dan keterampilan Anda.

1. Jelaskan fungsi dari tag `{% csrf_token %}` dan mengapa tag tersebut wajib ada di dalam form yang menggunakan metode `POST`.
2. Apa perbedaan utama dalam logika *view* antara `post_new` dan `post_edit` saat menangani permintaan `GET`?